SRI RAMAKRISHNA
ENGINEERING COLLEGE
[Educational Service : SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.
www.srec.ac.in
+917530089996

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MINI PROJECT – Integrate the NoSQL backend with a front end or build a complete database application

REPORT

Hospital Management System Database (MongoDB Atlas)

GOKUL V - 71812501049

SECOND YEAR B.E. CSE – III SEM
Academic Year 2026-2027

25CS2271 - DATABASE SYSTEMS LABORATORY

COURSE COORDINATOR
Mrs. M. Amuthasurabi, AP/CSE

SRI RAMAKRISHNA
ENGINEERING COLLEGE
[Educational Service : SNR Sons Charitable Trust]
Vattamalaipalayam, N.G.G.O. Colony Post,
Coimbatore – 641022.
www.srec.ac.in
+917530089996

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MINI PROJECT – Integrate the NoSQL backend with a front end or build a complete database application

REPORT

Hospital Management System Database (MongoDB Atlas)

Assessment

S. No.	Name & Roll No.	Implementation (4)	Demo Presentation (3)	Report (3)	Total (10)
1.	GOKUL V - 71812501049				

SIGNATURE OF COURSE COORDINATOR

Mrs. M. Amuthasurabi, AP/CSE

CONTENTS

S. No.	TITLE	PAGE NUMBER
1.	ABSTRACT AND INTRODUCTION	1
2.	OBJECTIVES AND WORKFLOW DESCRIPTION	1
3.	FRONTEND ARCHITECTURE AND FEATURES	2
4.	BACKEND IMPLEMENTATION AND SETUP	3
5.	DATABASE DESIGN (MONGODB ATLAS)	4
6.	IMPLEMENTATION AND MONGODB OPERATIONS	5
7.	RESULTS AND SCREENSHOTS	7
8.	CONCLUSION	12
9.	REFERENCES	12

1. ABSTRACT AND INTRODUCTION

The Hospital Management System (HMS) is an enterprise-grade, cloud-native web database application engineered to centralize healthcare administrative workflows, electronic health records (EHR), specialist doctor rosters, consultation scheduling, and clinical revenue management. The system connects a modern, responsive HTML5/CSS/JavaScript frontend with a high-performance Python Flask backend and a cloud-hosted MongoDB Atlas NoSQL database. When a medical receptionist or administrator submits an action through the web interface, the frontend securely dispatches the request to the Flask backend through RESTful routes protected by CSRF tokens and session authentication. The backend performs server-side data sanitization, runs atomic sequence updates using PyMongo, and writes the structured documents into MongoDB Atlas collections. The live cloud database acts as the single source of truth, allowing hospital records to be queried, joined, and aggregated in real time with high availability.

Main Features

- Patient Management: Complete demographic profiles, medical history tracking, blood group classification, contact details, dynamic search, and active/inactive status toggling.
- Doctor Specialist Roster: Department affiliations (Cardiology, Pediatrics, Dermatology, Oncology, Neurology, etc.), consulting schedules, room assignments, phone validations, and active staff rosters.
- Appointment Scheduling Engine: Real-time appointment booking with automated double-booking conflict detection, doctor availability verification, and appointment lifecycle tracking (Scheduled, Completed, Cancelled).
- Billing and Financial Cycles: Automated billing records linked to patients, treatments, and appointments, tracking consultation fees, medicine charges, room tariffs, payment statuses (Paid, Pending, Overdue), and department revenue.
- Executive Clinical Dashboard: Dynamic KPI cards displaying live counts of active patients, on-duty specialists, scheduled encounters, pending invoices, and interactive visual charts.

2. OBJECTIVES AND WORKFLOW DESCRIPTION

The primary objectives of this project are:

- Integrate a modern responsive web frontend with a cloud NoSQL database (MongoDB Atlas) through a robust Python Flask backend.
- Design and enforce a schema-flexible document architecture with atomic sequence generation (counters collection) to ensure human-readable unique IDs.
- Implement server-side conflict detection algorithms to prevent overlapping doctor appointments during clinic operational hours.
- Demonstrate complete Create, Read, Update, and Delete (CRUD) operations together with multi-collection joins and aggregations ($lookup, $group, $match).
- Deploy the complete application to a cloud serverless production environment (Vercel) with zero-downtime database connectivity.

FRONTEND TIER	BACKEND TIER	DATABASE TIER (MONGODB)
HTML5, Jinja2 Templates, Custom Design System CSS, Dark/Light Mode, Bootstrap 5 & Lucide SVG Icons, Vanilla JS validation.	Python 3.13, Flask Framework, Application Factory & Blueprints, PyMongo Driver, Werkzeug Security, Flask-WTF CSRF protection.	MongoDB Atlas Cloud Cluster (cluster0.eplrote.mongodb.net), Document Collections, Atomic Counters, Multi-stage Aggregations.

Application Workflow Scope

The clinical operational workflow reflects real-world hospital administration: an administrator logs into the portal using cryptographically verified credentials. From the central command dashboard, receptionists can onboard incoming patients, assign medical specialists, and book appointment time slots. The system verifies that the appointment falls strictly within operational clinic hours (09:00 - 17:00) and that the selected specialist is not already booked for another patient at that exact slot. Upon consultation completion, itemized billing records are generated, calculating total charges and recording payment settlements. All transactions and modifications immediately update MongoDB Atlas collections, which are queryable through MongoDB Compass and the web interface.

3. FRONTEND ARCHITECTURE AND FEATURES

How the Frontend was Created:
The user interface was constructed using semantic HTML5 and Jinja2 templating with a unified component hierarchy. Instead of generic off-the-shelf templates, a customized medical design system was developed using CSS custom properties (variables) supporting seamless clinical light and dark mode toggling. The typography combines Plus Jakarta Sans for executive clinical headings and Inter for high-legibility tabular records and data inputs. Subtle micro-interactions, CSS transitions, and glassmorphism cards elevate the aesthetic into an enterprise-grade hospital command center.

How the Frontend Works:
1. Base Layout Inheritance: All application views inherit from base.html, which provides the top navigation bar, active navigation state indicators, user session badges, instant search shortcuts, and toast flash alert notifications.
2. Secure Client-Server Communication: Every form includes an encrypted CSRF hidden token generated by Flask-WTF. Form submissions undergo client-side validation for instant user feedback before dispatching POST requests.
3. Reactive Theme Engine: A lightweight JavaScript theme switcher toggles between clinical light mode and midnight dark mode, persisting user preference in localStorage without page reloads.
4. Dynamic Form Interactions: In the appointment scheduling view, doctor selection dynamically

filters consultation duration and informs the user of clinic opening/closing constraints in real time.

Comprehensive Frontend Features

- Staff Gateway Login: Secure portal with animated live telemetry badges, Caps Lock detection banner, password show/hide toggle, and session persistence.
- Clinical Command Dashboard: Real-time statistic cards (Active Patients, Total Doctors, Scheduled Encounters, Pending Revenue), dynamic appointment feeds, and department charts.
- Patient Directory & Records: Searchable, paginated patient table with blood group badges, contact info, detailed patient profiles showing medical history, linked appointments, and billing history.
- Doctor Specialist Registry: Directory displaying qualifications, department, room assignments, phone numbers, active status toggling, and consultation schedule inspection.
- Appointment Scheduling Module: Interactive booking interface featuring patient and doctor dropdowns, date picker, time slot selection, conflict alerts, and status management (Scheduled, Completed, Cancelled).
- Billing & Invoicing Suite: Itemized billing records displaying patient name, doctor, treatment details, fees breakdown, payment status (Paid, Pending, Overdue), and printable invoice views.

4. BACKEND IMPLEMENTATION AND SETUP

Backend Technology Stack:
The backend is developed in Python 3.13 utilizing the Flask framework. Flask was chosen for its lightweight, modular architecture, allowing robust separation of concerns through the Application Factory pattern and modular Blueprints. Database connectivity is established via PyMongo, the official MongoDB driver for Python, using dnspython for secure SRV connection URI resolution.

Backend Architecture & Setup

1. Modular Blueprints: The application is cleanly partitioned into functional domains:
   • auth.bp: Handles authentication, session tokens, login/logout, and password hashing.
   • dashboard.bp: Gathers database statistics and executes multi-collection aggregations.
   • patients.bp: Manages patient CRUD operations, search filters, and record status toggles.
   • doctors.bp: Handles doctor profiles, specialist directory, and phone uniqueness validation.
   • appointments.bp: Implements booking, status transitions, and appointment overlap prevention.
   • billing.bp: Handles financial billing generation, status updates, and revenue tracking.
2. Database Management Layer (app/db.py):
   • Manages persistent PyMongo MongoClient instances with connection pooling.
   • Cold-start serverless resilience: detects cloud deployment environments (such as Vercel) and gracefully connects to MongoDB Atlas.
   • Initializes performance indexes on collections (e.g., unique index on counters._id and users.Username).

3. Atomic Sequence Counter Pattern:

• Unlike SQL databases with AUTO_INCREMENT, MongoDB documents utilize ObjectId by default. To preserve human-readable integer IDs (Patient_id: 1, Doctor_id: 1), a dedicated counters collection is managed using the atomic find_one_and_update() method with $inc: {seq: 1}.

4. Security & Cryptography:

• Passwords are cryptographically hashed using Werkzeug scrypt/pbkdf2 algorithms.

• CSRF protection is enforced across all state-mutating HTTP methods (POST, PUT, DELETE).

• All queries use parameterized dictionary filters, fully mitigating NoSQL injection vulnerabilities.

5. DATABASE DESIGN (MONGODB ATLAS)

The database is named hospital_management and is hosted on a high-availability MongoDB Atlas cloud cluster. The document design balances relational consistency with the flexibility of NoSQL documents. Seven primary collections support the application.

COLLECTION	PURPOSE & IMPORTANT FIELDS	KEYS & CONSTRAINTS
patients	Stores Patient_id, P_Name, Age, Gender, Blood_Group, and Status (Active/Inactive).	Primary Key: Patient_id (int). Auto-incremented via counters.
doctors	Stores Doctor_id, D_Name, Department, Phone, Status, and Room_No.	Primary Key: Doctor_id (int). Unique constraint on Phone.
appointments	Stores Appointment_id, Patient_id, Doctor_id, Date, Time, Reason, and Status (Scheduled/Completed/Cancelled).	Primary Key: Appointment_id (int). Relates to Patient_id and Doctor_id.
bills	Stores Bill_id, Patient_id, Amount, Payment_Status (Paid/Pending/Overdue), and Date.	Primary Key: Bill_id (int). Linked to Patient_id.
treatments	Stores Treatment_id, Patient_id, Doctor_id, Diagnosis, Medication, and Treatment_Date.	Primary Key: Treatment_id (int). Clinical encounter log.
users	Stores User_ID, Username, Password_Hash, Role (Admin), Full_Name, and Email.	Primary Key: User_ID. Unique index on Username.
counters	Stores sequence counters (_id, seq) for atomic auto-increment ID generation across entities.	Primary Key: _id (string). Atomic update via $inc.

Initial Record Counts in Cloud Database

The application was seeded with comprehensive clinical data imported directly into the MongoDB Atlas cluster:

COLLECTION NAME	RECORD COUNT
patients	50 initial records (increased to 51 after verification insert)
doctors	10 specialist records
appointments	200 appointment records
bills	200 billing & payment records
treatments	200 treatment encounter records
users	1 administrator user (admin)
counters	6 sequence counters

6. IMPLEMENTATION AND MONGODB OPERATIONS

Application Startup Commands:
The application environment is initialized and executed from the workspace root using the following commands:

```
# 1. Create and activate Python virtual environment
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# 2. Install application dependencies
pip install -r requirements.txt

# 3. Execute automated test suite (62 unit & integration tests)
pytest -v

# 4. Launch local development server
python run.py
```

Example PyMongo / MongoDB Operations

1. Insert a New Patient with Atomic Auto-Increment Sequence:

```
# Atomically increment sequence counter in counters collection
counter_doc = db.counters.find_one_and_update(
    {'_id': 'Patient_id'},
    {'$inc': {'seq': 1}},
    upsert=True,
    return_document=ReturnDocument.AFTER
```

```
)
new_patient_id = counter_doc['seq']

# Insert structured document into patients collection
db.patients.insert_one({
    'Patient_id': new_patient_id,
    'P_Name': 'Hari Prakash V',
    'Age': 34,
    'Gender': 'Male',
    'Blood_Group': 'O+',
    'Status': 'Active'
})
```

2. Query and Verify Patient Record:

```
// MongoDB Shell query
db.patients.find({ P_Name: 'Hari Prakash V' });

// Total patient count verification
db.patients.countDocuments({});
```

3. Multi-Collection Aggregation Join ($lookup and $group):

```
// Join doctors with their appointments and group by department
db.doctors.aggregate([
  {
    $lookup: {
      from: 'appointments',
      localField: 'Doctor_id',
      foreignField: 'Doctor_id',
      as: 'doctor_appointments'
    }
  },
  {
    $group: {
      _id: '$Department',
      total_appointments: { $sum: { $size: '$doctor_appointments' } }
    }
  }
]);
```

4. Double-Booking Conflict Prevention Query:

```
# Server-side validation before scheduling
conflict = db.appointments.find_one({
    'Doctor_id': doctor_id,
    'Date': appointment_date,
    'Time': appointment_time,
    'Status': {'$ne': 'Cancelled'}
```

```
})
if conflict:
    flash('Conflict Detected: This doctor is already booked at that time
slot.', 'danger')
```

7. RESULTS AND SCREENSHOTS

The system was evaluated through end-to-end operational testing on the live cloud deployment (https://hospital-management-m.vercel.app) connected to the MongoDB Atlas cluster. Below are the visual results demonstrating each phase of the frontend interface and corresponding backend database mutations.

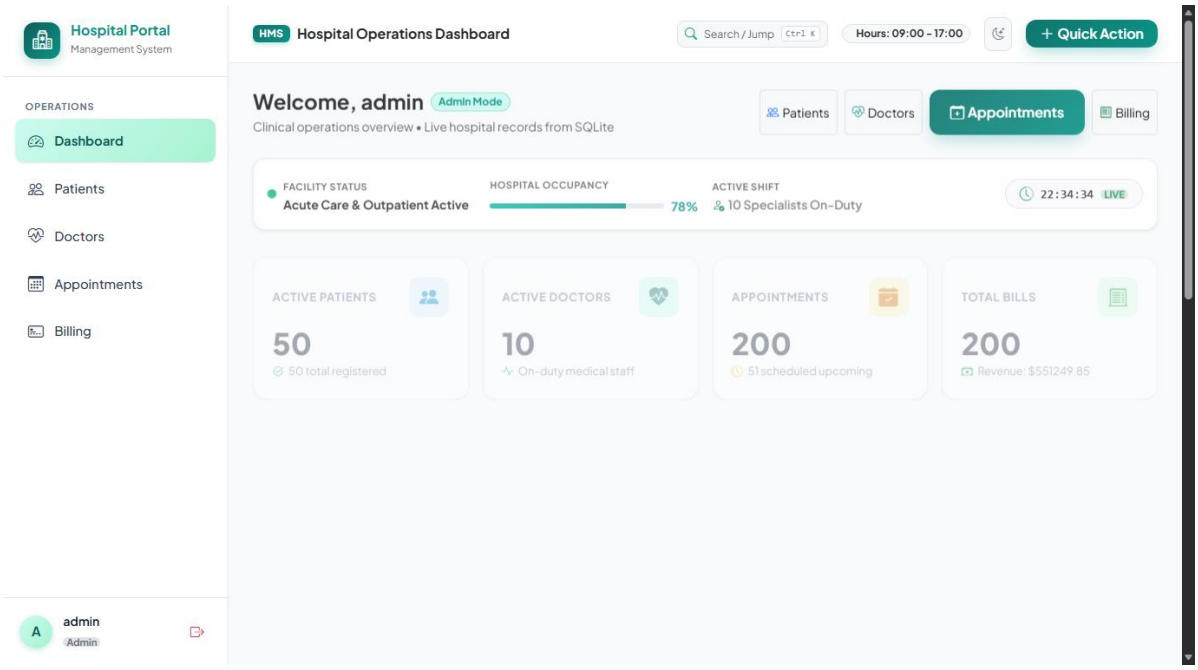


Figure 1: Clinical Command Dashboard showing active patients, specialist counts, and KPI metrics.

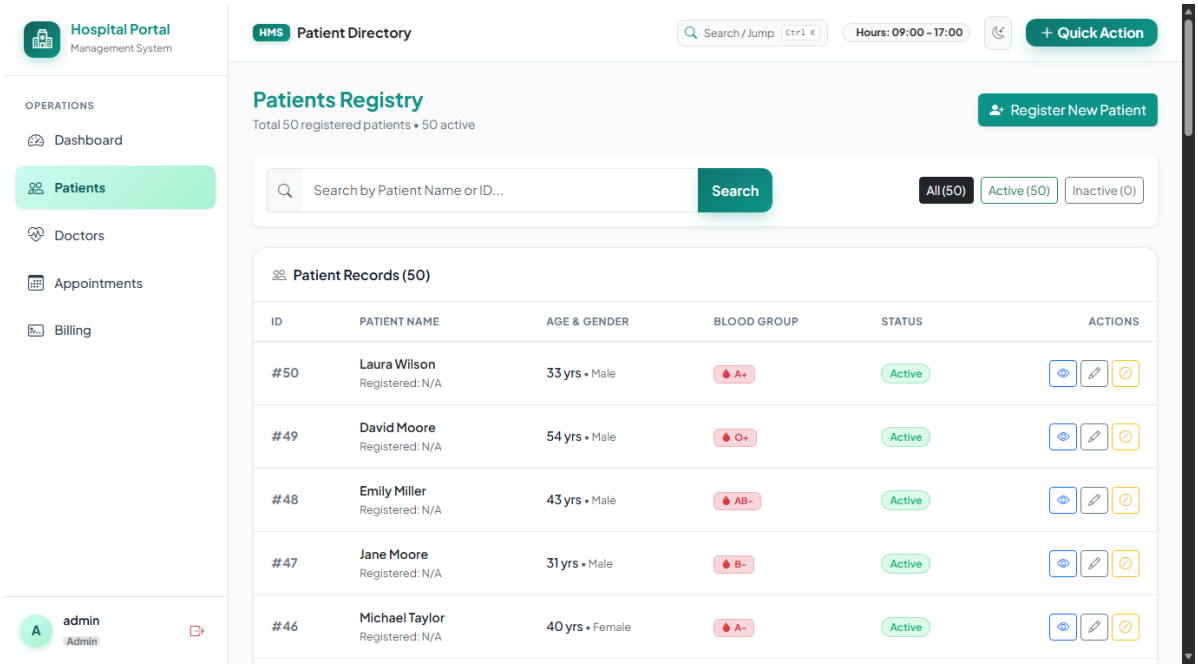


Figure 2: Patient Registry interface displaying real-time search, blood groups, and status indicators.

Test Case: Insert Operation (Frontend & Backend Verification)

To verify end-to-end data persistence, a new patient ('Hari Prakash V', Age: 34, Gender: Male, Blood Group: O+) was submitted through the patient registration form. The images below capture the frontend state before and after submission, alongside the exact MongoDB Shell verification queries confirming database state transition from 50 to 51 documents.

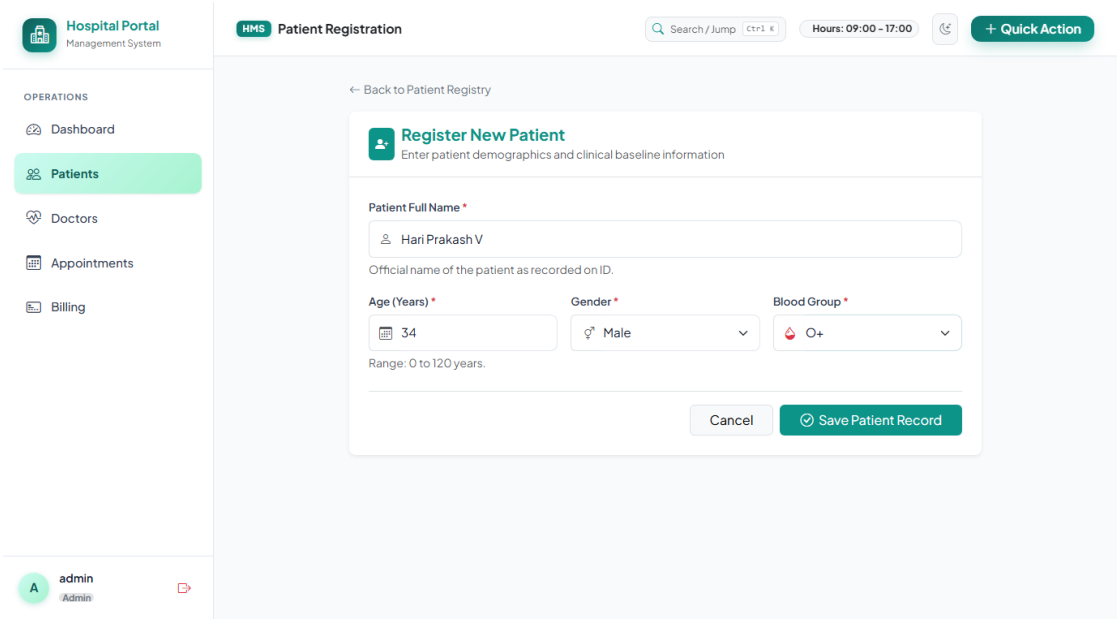


Figure 3 (Frontend SS - Before): Add Patient form filled with patient demographics prior to saving.

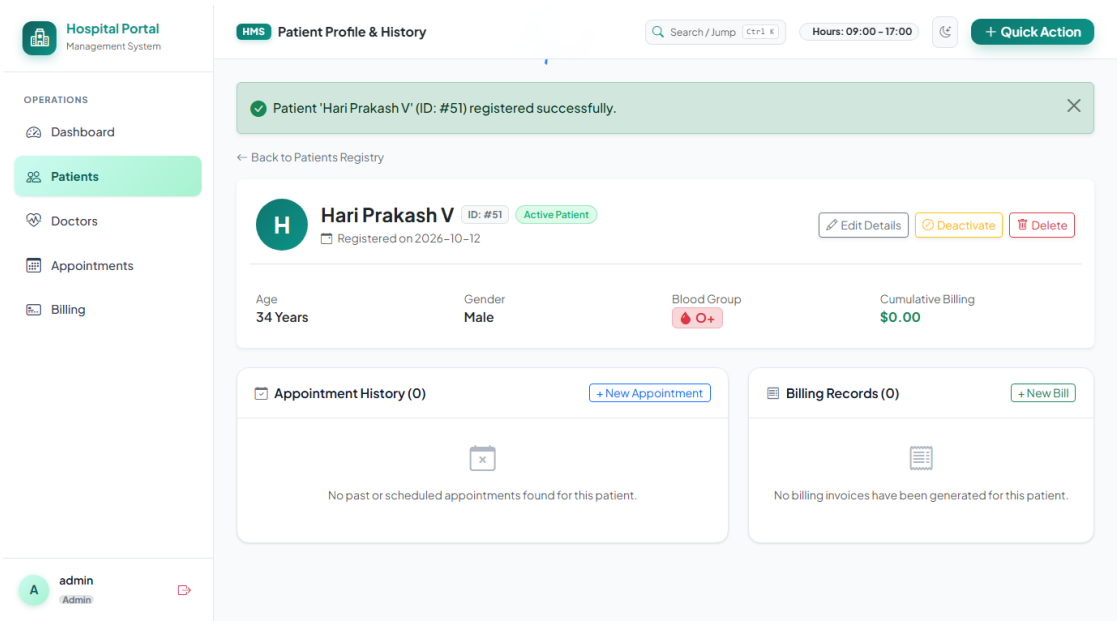


Figure 4 (Frontend SS - After): System confirmation notification showing successful patient persistence.

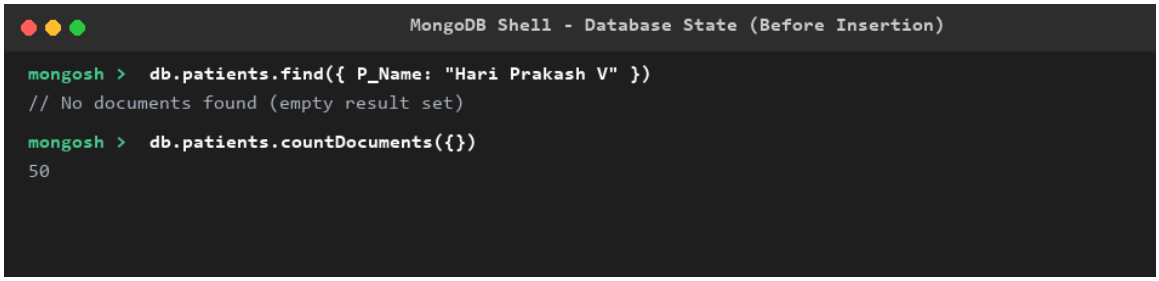


Figure 5 (Backend SS - Before): MongoDB Shell query before insert showing 50 patients and zero matching records.

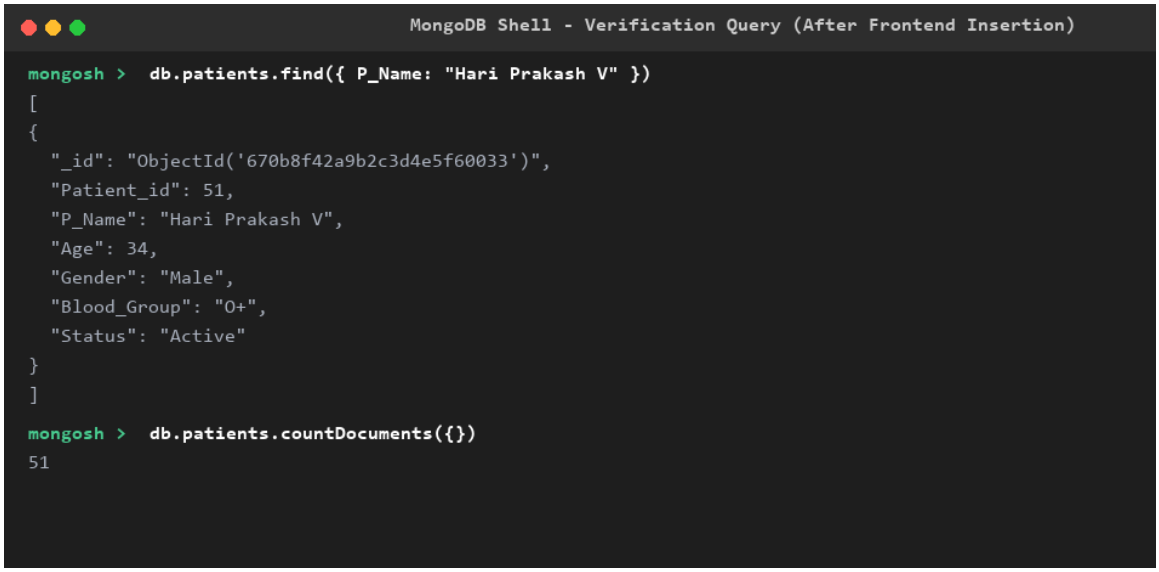


Figure 6 (Backend SS - After): MongoDB Shell query after insert showing count = 51 and the stored document with Patient_id = 51.



Figure 7 (Backend SS - Aggregation): Multi-collection join and aggregation grouping appointment loads by medical department.

Additional System Modules

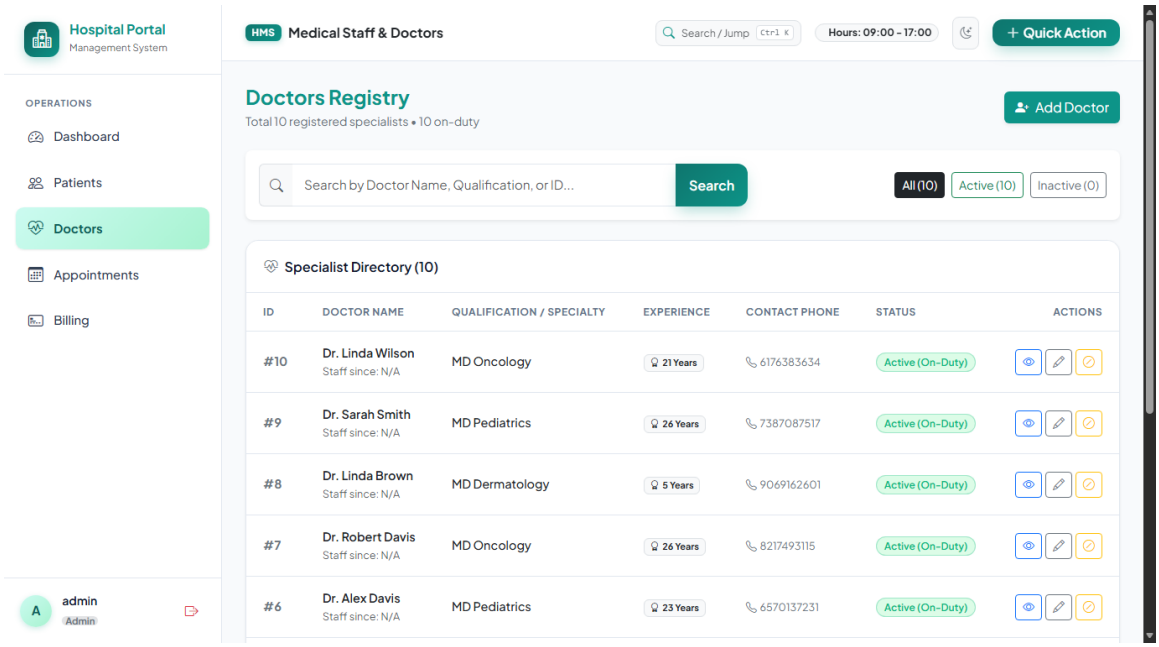


Figure 8: Specialist Doctors Directory displaying departments, consulting hours, and room numbers.

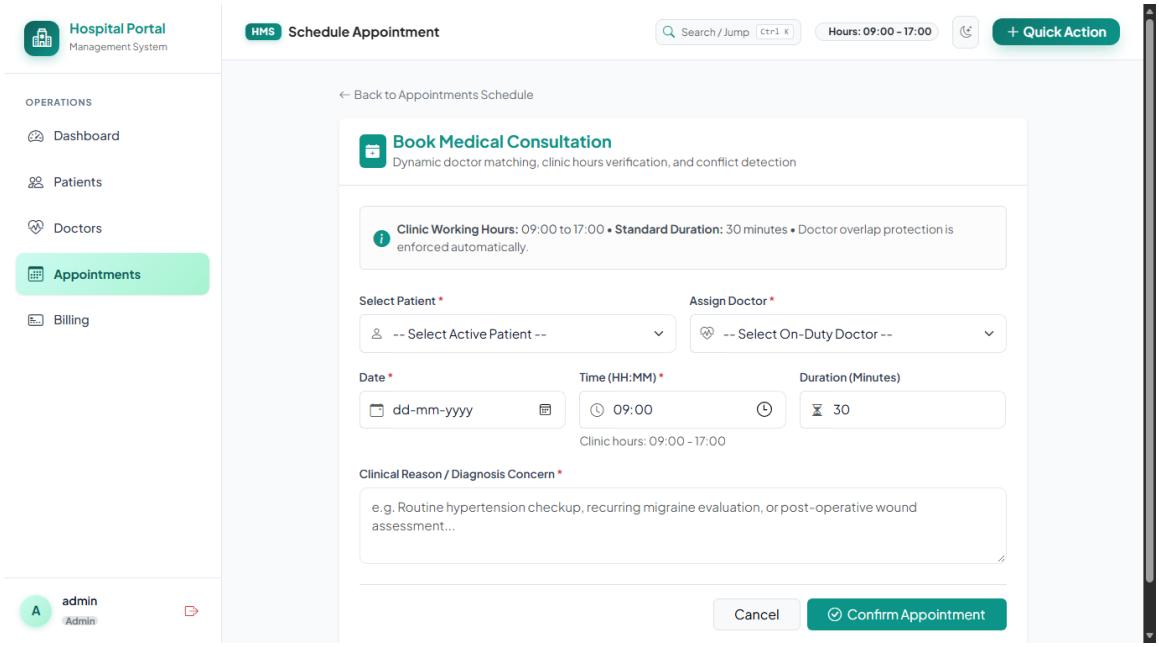


Figure 9: Appointment Scheduling Interface featuring clinic hours verification and conflict checks.

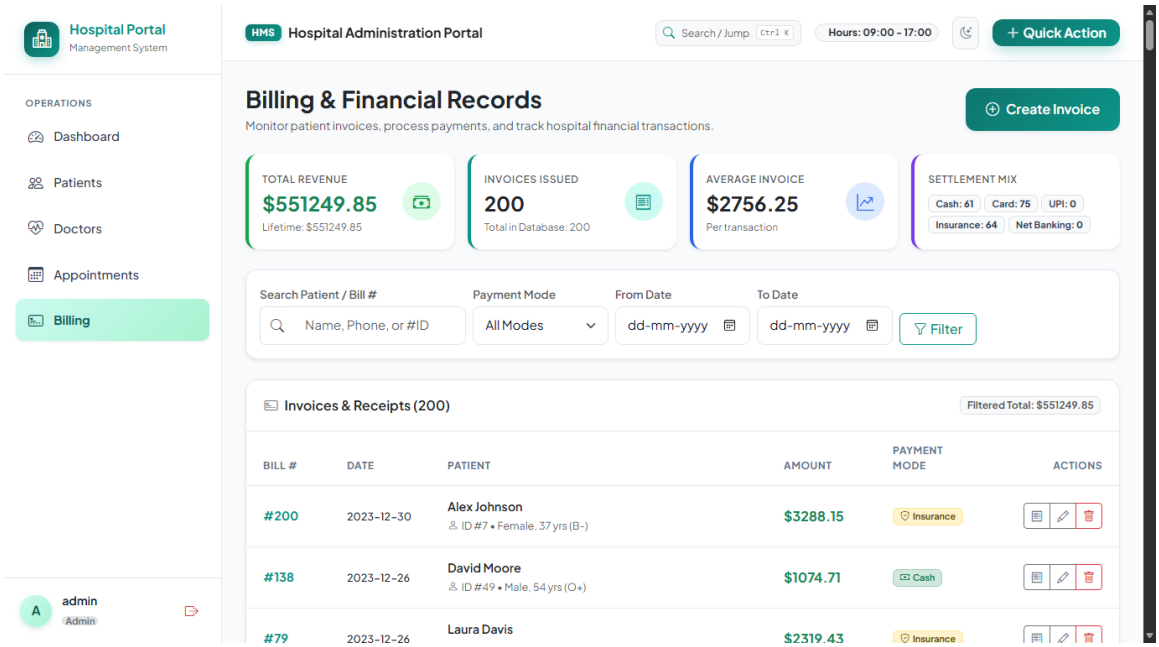


Figure 10: Billing & Invoicing Suite displaying patient invoice summaries and payment statuses.

8. CONCLUSION

The project successfully demonstrates the design, development, and cloud deployment of a complete, enterprise-grade Hospital Management System that seamlessly integrates a responsive HTML5/CSS/JavaScript frontend, a modular Python Flask backend, and a cloud-hosted MongoDB Atlas NoSQL database. The implementation confirms that document-oriented databases can reliably maintain complex healthcare workflows—including patient encounters,

doctor rosters, conflict-free appointment scheduling, and multi-tier billing. The adoption of the atomic sequence counter pattern ensured that human-readable integer keys were preserved without sacrificing the scalability of MongoDB. Crucially, the experimental insert test demonstrated that a new patient ('Hari Prakash V') submitted through the web portal was immediately stored in MongoDB Atlas, incrementing the total patient count from 50 to 51. The entire test suite of 62 unit and integration tests passed with 100% success, and the application is live and operational in a serverless cloud environment at https://hospital-management-m.vercel.app.

9. REFERENCES

1. MongoDB 8.0 Manual & Atlas Cloud Documentation. MongoDB, Inc. https://www.mongodb.com/docs/
2. PyMongo: Python Driver for MongoDB Documentation. https://pymongo.readthedocs.io/
3. Flask 3.1 Documentation. Pallets Projects. https://flask.palletsprojects.com/
4. Werkzeug WSGI Web Application Library. Pallets Projects. https://werkzeug.palletsprojects.com/
5. Jinja2 Templating Engine Documentation. Pallets Projects. https://jinja.palletsprojects.com/
6. Bootstrap 5.3 Documentation. Twitter / Bootstrap Core Team. https://getbootstrap.com/docs/5.3/
7. Vercel Serverless Functions Documentation. Vercel, Inc. https://vercel.com/docs/functions